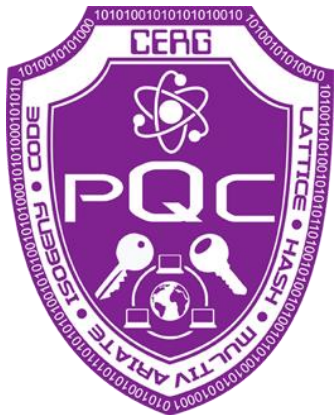


High-Performance Hardware Implementation of Lattice-Based Digital Signatures

Luke Beckwith, Duc Tri Nguyen, Kris Gaj

George Mason University
USA



Contributions and Outline

Summary of Contributions:

- Publicly available high-performance hardware implementation of Dilithium which supports all modes and security levels
- First hardware implementation of a complete FALCON operation

Outline:

- Challenges of implementing Dilithium in hardware
- Our hardware implementation of Dilithium and FALCON-verify
- Results and comparisons

Challenges of Dilithium in Hardware

Challenges of Dilithium in Hardware

- **Polynomial sampling** can easily bottleneck performance
- **Number Theoretic Transform** accelerates polynomial multiplication, but is complex to implement efficiently
- Dilithium **signature rejection** may require multiple attempts to generate securely

Dilithium Performance Bottlenecks

Polynomial sampling

- Core elements are matrices and vectors of polynomials
- Sampled uniformly from output of SHA-3 extendable-output function (XOF) SHAKE

$$\begin{matrix} & \overbrace{\hspace{1.5cm}}^l \\ \underbrace{\hspace{1cm}}_k \left\{ \begin{bmatrix} A_{1,1} & \cdots & A_{1,l} \\ \vdots & \ddots & \vdots \\ A_{k,1} & \cdots & A_{k,l} \end{bmatrix} \right. \end{matrix}$$

$$A_{i,j} \in R_q = a_{N-1}x^{N-1} + \dots + a_1x^1 + a_0$$

where R_q is the ring $Z_q[X]/(X^N + 1)$

Dilithium Performance Bottlenecks

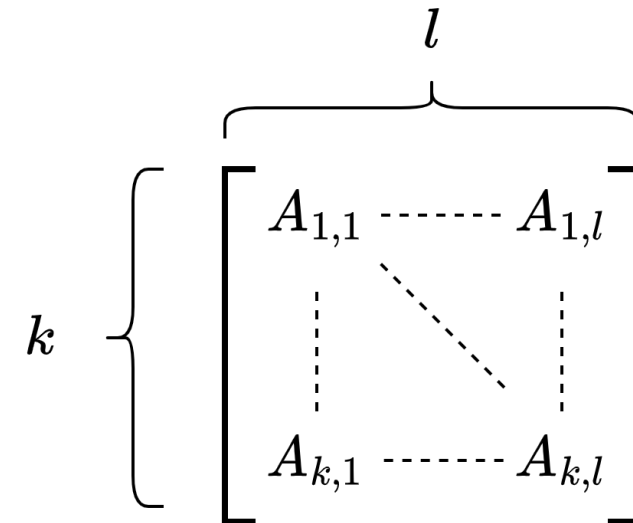
Polynomial sampling

- Matrix/vector dimensions determined by security level
- Large matrices → **lots of pseudorandom data**

Example: Bytes needed to sample matrix for Dilithium5:

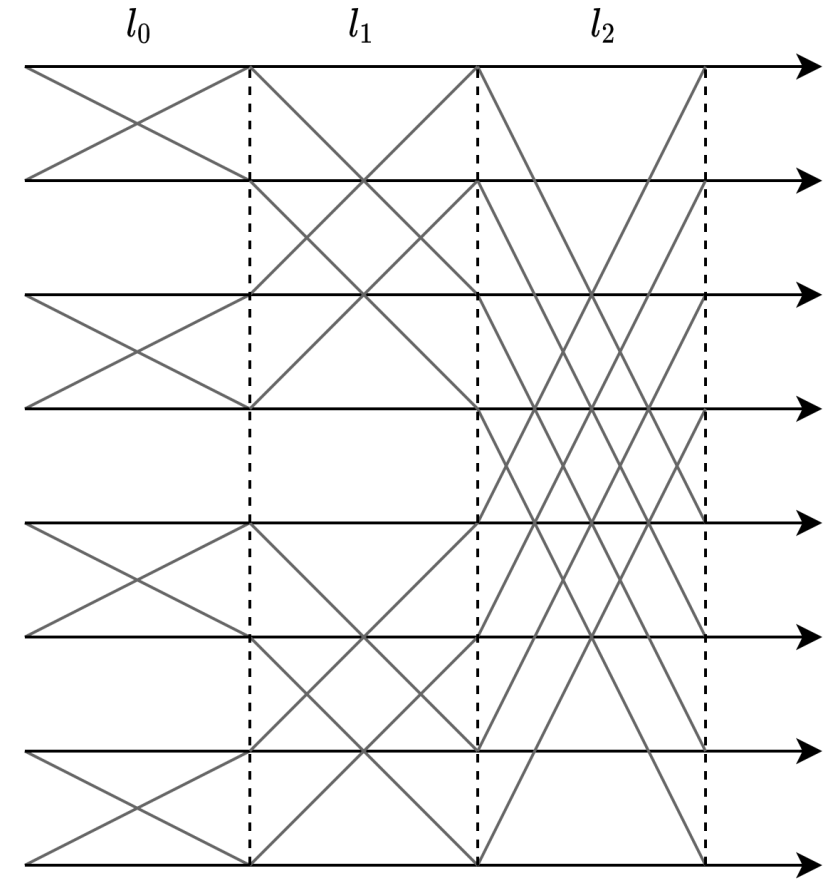
$$\frac{\text{bytes}}{\text{coeff}} * N * k * l = 3 * 256 * 8 * 7 = 43KB$$

Parameter	Dilithium2	Dilithium3	Dilithium5
Dimensions (k, l)	(4, 4)	(6, 5)	(8, 7)
Coefficient Modulus (q)	$8,380,417 = 2^{23} - 2^{13} + 1$		
Number of Polynomial Coefficients (N)	256		



Number Theoretic Transform

- Used for polynomial multiplication
 - Reduces complexity from $O(N^2)$ to $O(N \log(N))$
 - $a * b = NTT^{-1}(NTT(a) \circ NTT(b))$
- $\log_2(N)$ layers
- $N/2$ butterfly operations per layer
 - Cooley-Tukey Butterfly (Forward):
 - $a' = a + \omega \cdot b \bmod q$
 - $b' = a - \omega \cdot b \bmod q$
 - Gentleman-Sande Butterfly (Inverse):
 - $a' = a + b \bmod q,$
 - $b' = (a - b) \cdot \omega \bmod q$
- Inverse also requires scaling by $\frac{1}{N} = \left(\frac{1}{2}\right)^{\log_2 N}$



“Butterfly” shape of an 8-point NTT

Dilithium Performance Bottlenecks

Matrix by vector multiplication

- Number Theoretic Transform (NTT) based multiplication improves performance **but is still expensive**
- Matrix is sampled in NTT-domain, but input vector must be converted using the NTT and result must be converted back using Inverse NTT

Example: Matrix multiplication requires l NTT transformations, k inverse NTT transformations, $k * l$ point-wise multiplications, and $k * (l - 1)$ additions

$$\begin{matrix} & & l \\ & \underbrace{\hspace{1.5cm}} & \\ k \left\{ \begin{bmatrix} \vdots \\ t \\ \vdots \end{bmatrix} \right. & = & \begin{bmatrix} \vdots \\ \vdots \\ \vdots \\ A \\ \vdots \\ \vdots \end{bmatrix} \times \begin{bmatrix} \vdots \\ s_1 \\ \vdots \end{bmatrix} \end{matrix}$$

Dilithium Performance Bottlenecks

Signature Rejection:

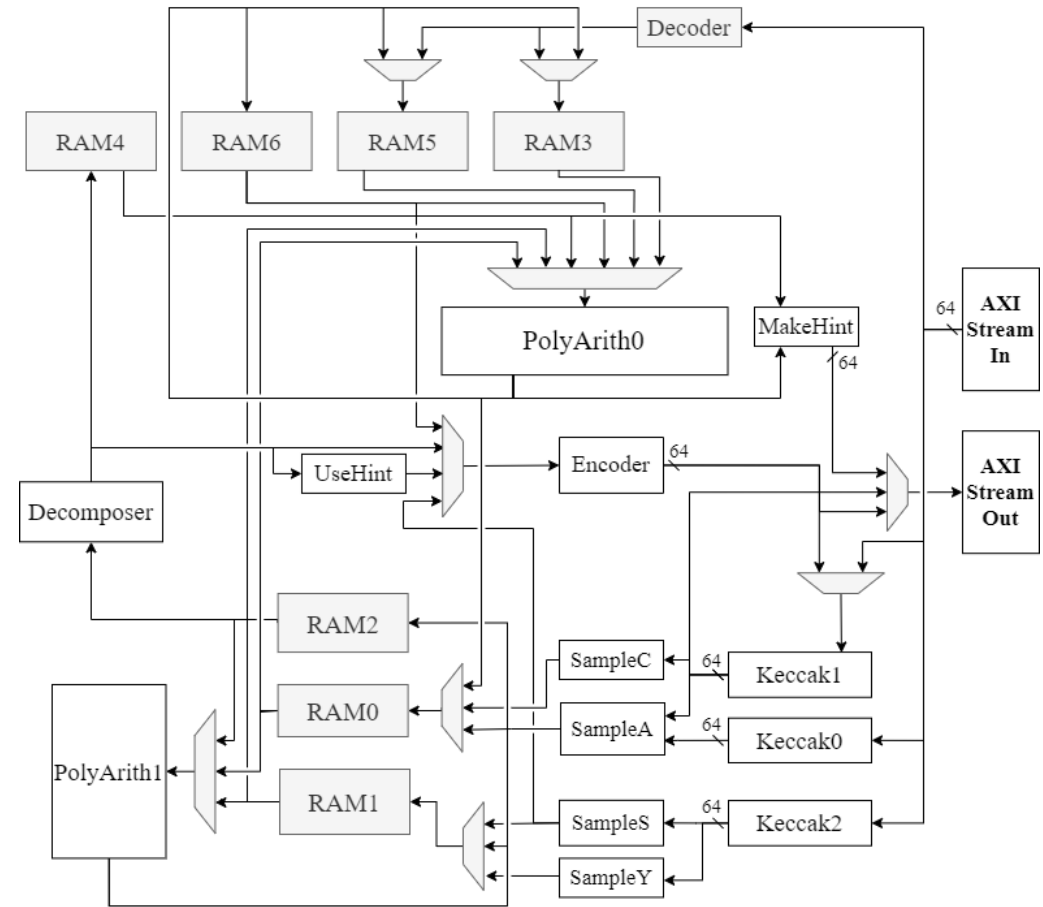
- Some signature attempts may leak information about secret key and must be rejected
- Rejection based on pre-defined conditions for acceptable bounds of signature polynomials and signature hint
- Roughly **4-5 attempts** needed depending on security level

Parameter	Dilithium2	Dilithium3	Dilithium5
Average Repetitions	4.25	5.1	3.85

Dilithium Hardware Design

Hardware Architecture – Sampling Bottleneck

- Reduce sampling latency by using 3 Keccak cores
 - 2 cores for matrix sampling
 - 1 for vectors and hashing
- **Allows most sampling to be hidden behind polynomial arithmetic**

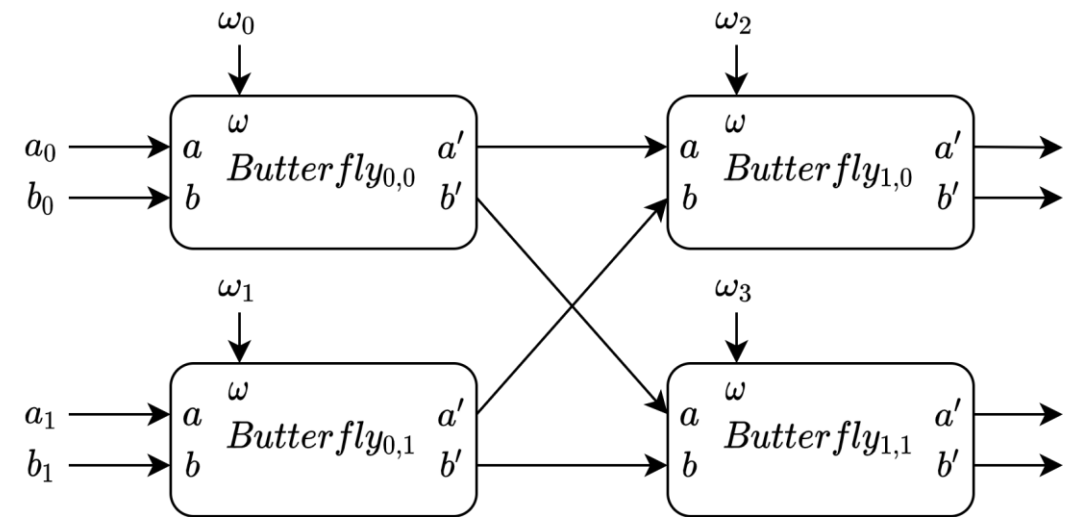


Default Bus: 96-bit

Hardware Architecture – NTT Bottleneck

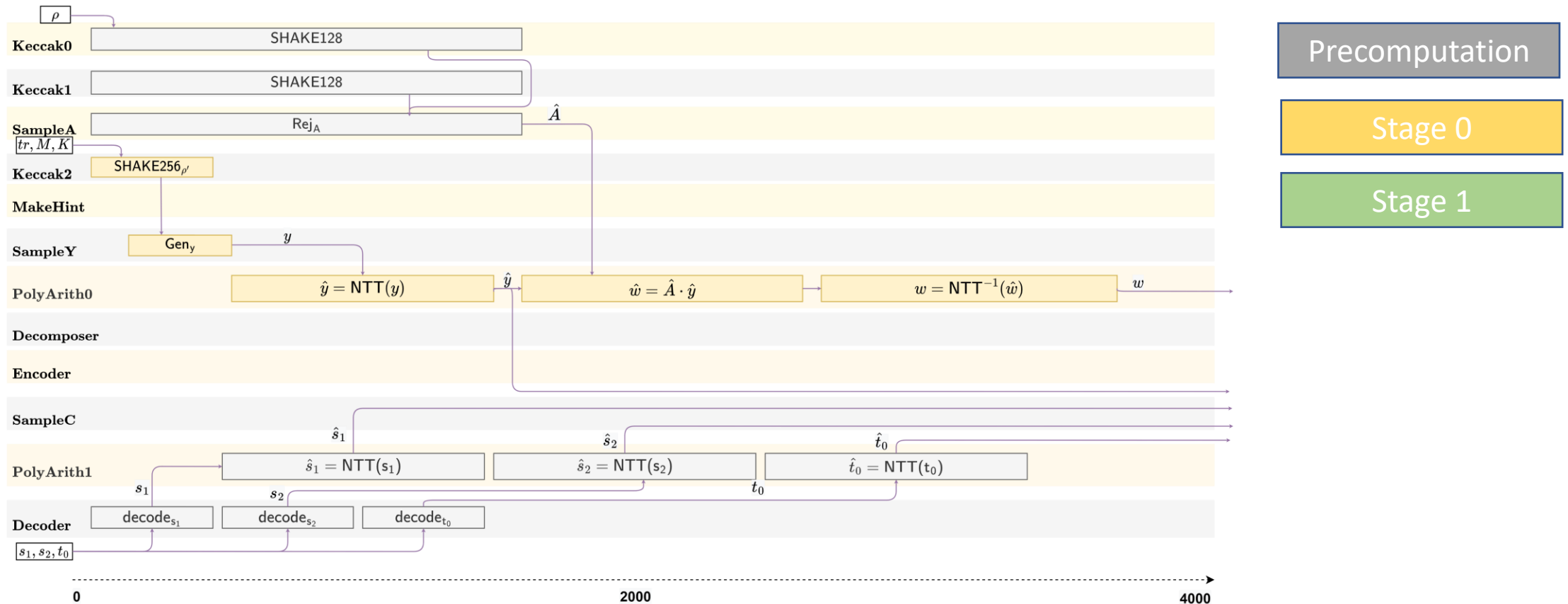
- 2x2 Butterfly:
 - Calculate two layers at a time to reduce memory access
 - Reorder coefficients to avoid conflicts
- Butterfly (BF) logic is reused for basic polynomial arithmetic (add/sub/point-wise multiply)
- Latency (cycles):

$$(\log_2(N) \text{ layers}) * \frac{\frac{N}{2} \text{ BF ops}}{1 \text{ layer}} * \frac{1 \text{ input}}{4 \text{ BF ops}} * \frac{1 \text{ cycle}}{\text{input}} + \text{depth}$$
$$= 256 + \text{depth}$$



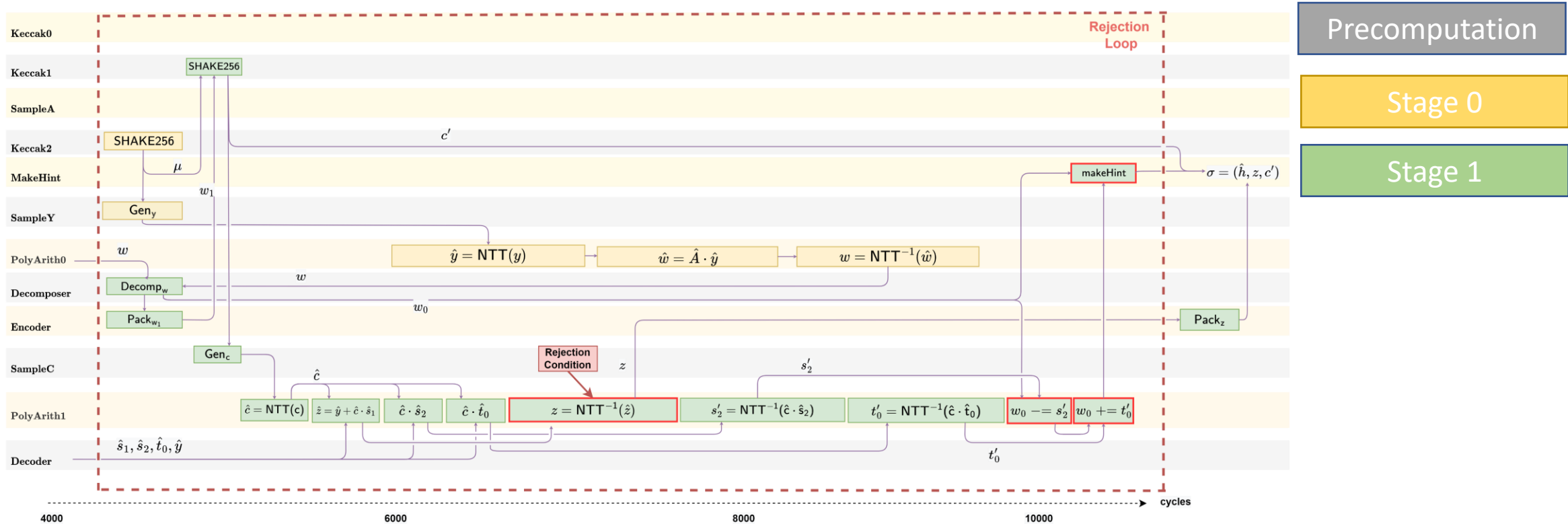
Default Bus: 23-bit

Dilithium Hardware Scheduling – Sign



- Signature rejection split into two stages
- Precomputation runs once, Stage 0 and Stage 1 run in parallel until complete

Dilithium Hardware Scheduling – Sign



- Stage 0 and Stage 1 run in parallel until complete
- Rejection conditions evaluated by PolyArith1 and makeHint

FALCON Hardware Design

FALCON Verify Overview

- **NTRU Lattice-Based Scheme**

- No previous full HW works
- Keygen and Sign are complicated
 - Uses complex trapdoor sampler requiring floating point FFT
- **Verify is much simpler!**
 - All elements are polynomials with integer coefficients
 - Security scales with number of polynomial coefficients

Parameter	FALCON-512	FALCON-1024
Coefficient Modulus (q)	$12,289 = 12 * 2^{10} + 1$	
Number of Polynomial Coefficients (N)	512	1024

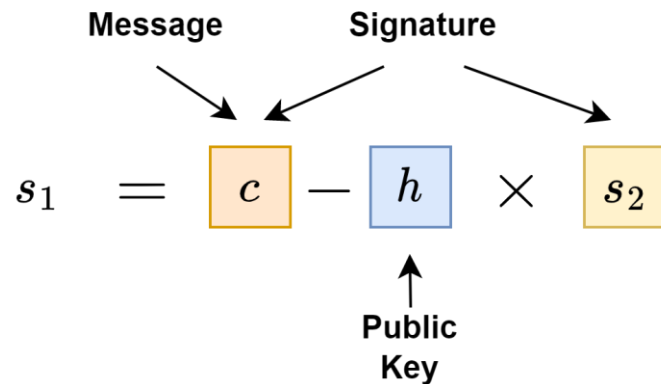
FALCON Verify Overview

Verify Operation:

Given message M , signature ($r \in \{0,1\}^{320}, s_2 \in Z_q[x]$), and the public key ($h \in Z_q[x]$), calculate:

$$c \leftarrow \text{HashToPoint}(r||M)$$
$$s_1 = c - h * s_2$$

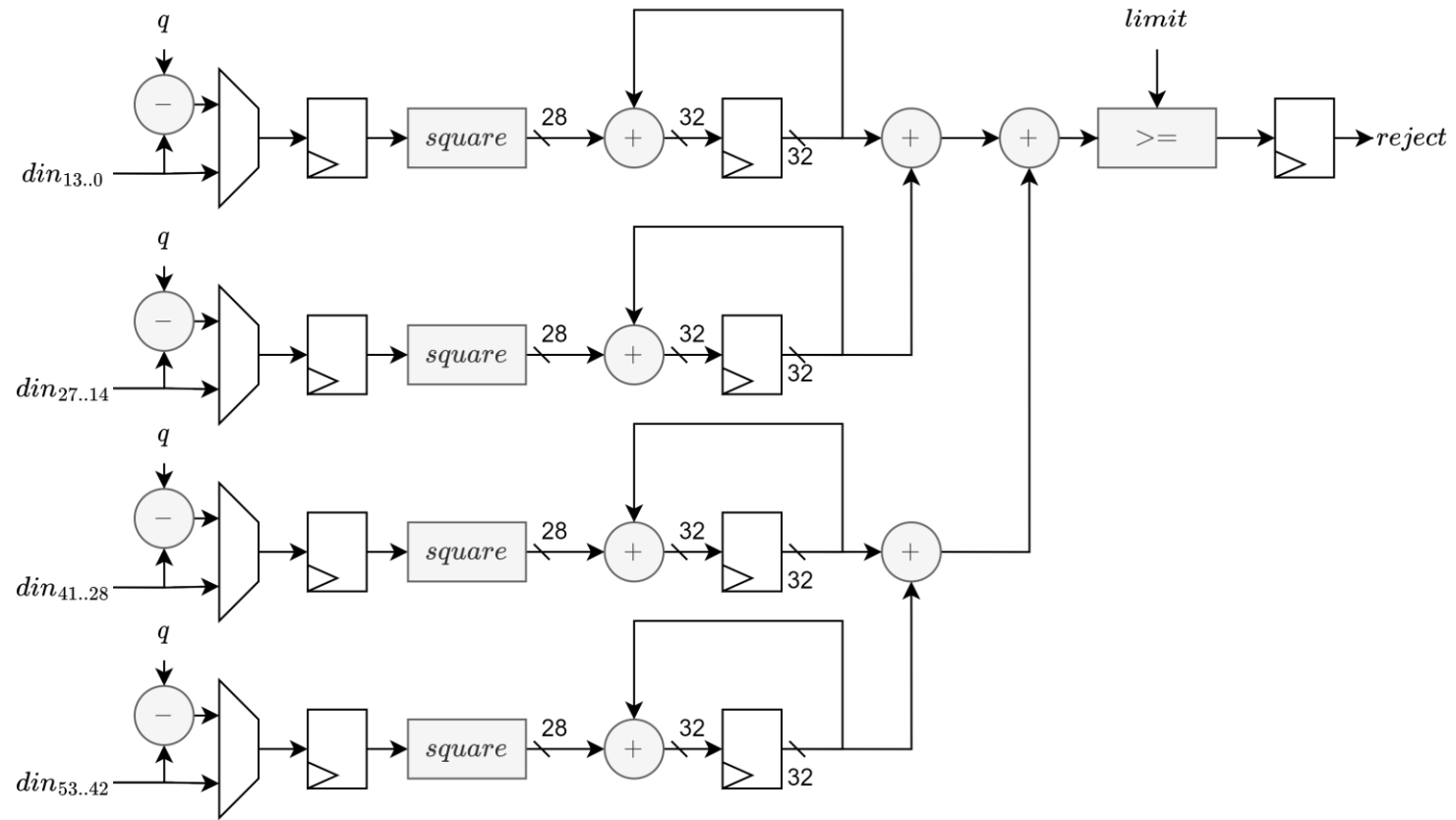
If $||s_1, s_2||^2 = \sum_{i=0}^{n-1} (s_1^2[i] + s_2^2[i])$ is less than the parameter value β , accept



FALCON Design Approach

- Similar structure to Dilithium – **some modules can be easily reused!**
 - NTT design (with minor modification)
 - Decoder for unpacking byte-packed public key
 - Rejection sampler
- Some new modules are required:
 - Norm
 - Decompression

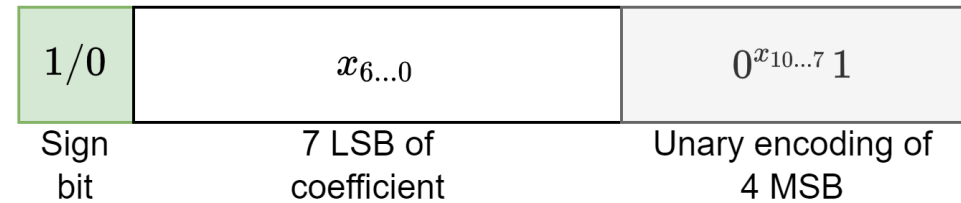
FALCON Norm



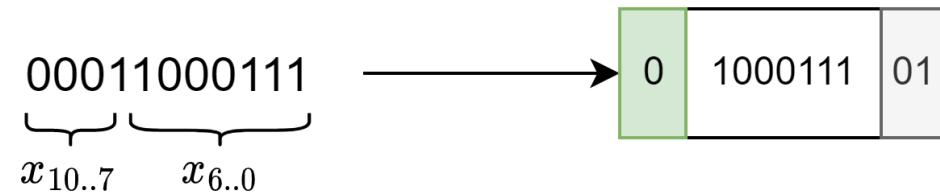
Norm Architecture

FALCON Compression

- Compression of Gaussian distribution samples
 - Encode as absolute value, append sign bit
 - Least significant bits are uniform - encode directly
 - Most significant bits are likely to be small - use unary encoding

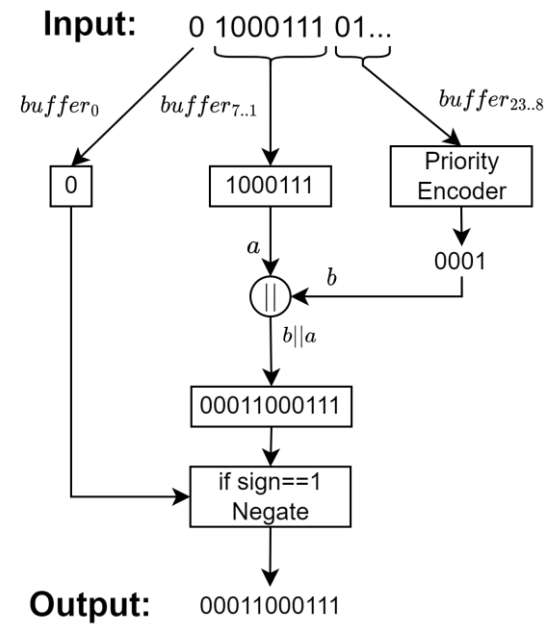


Compression Format

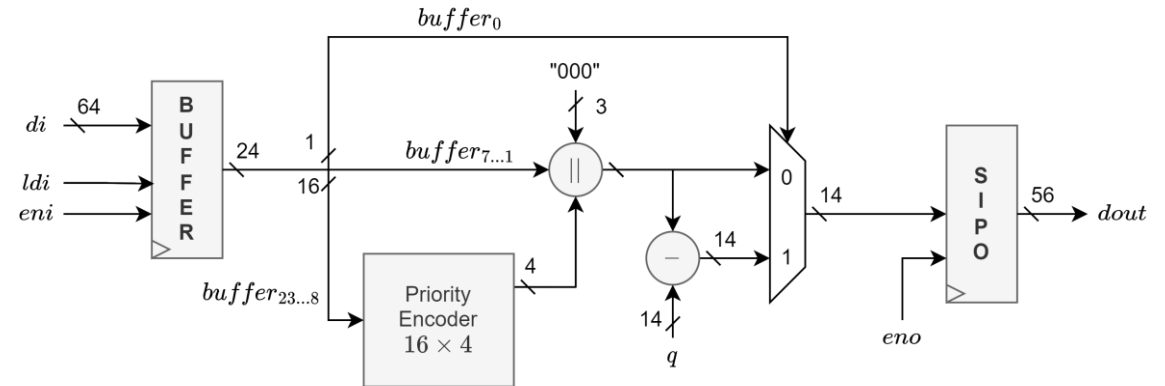


Example Compression

FALCON Decompression



Decompression Example



Decompressor Architecture

Results and Comparison

Results and Comparison

- **This work:**
 - Dilithium was implemented fully in Verilog except for preexisting Keccak implementation in VHDL
 - FALCON-Verify was implemented fully in VHDL except for components converted from Dilithium
 - Results generated using Vivado 2020.2
 - Maximum frequencies determined using the Minerva optimization tool
- **Comparison Assumptions:**
 - **Artix-7 FPGA:** Level 5 shown for most complete results and fair comparison with other Dilithium implementations
 - **Kintex-7 FPGA:** Level 5 shown for most complete results

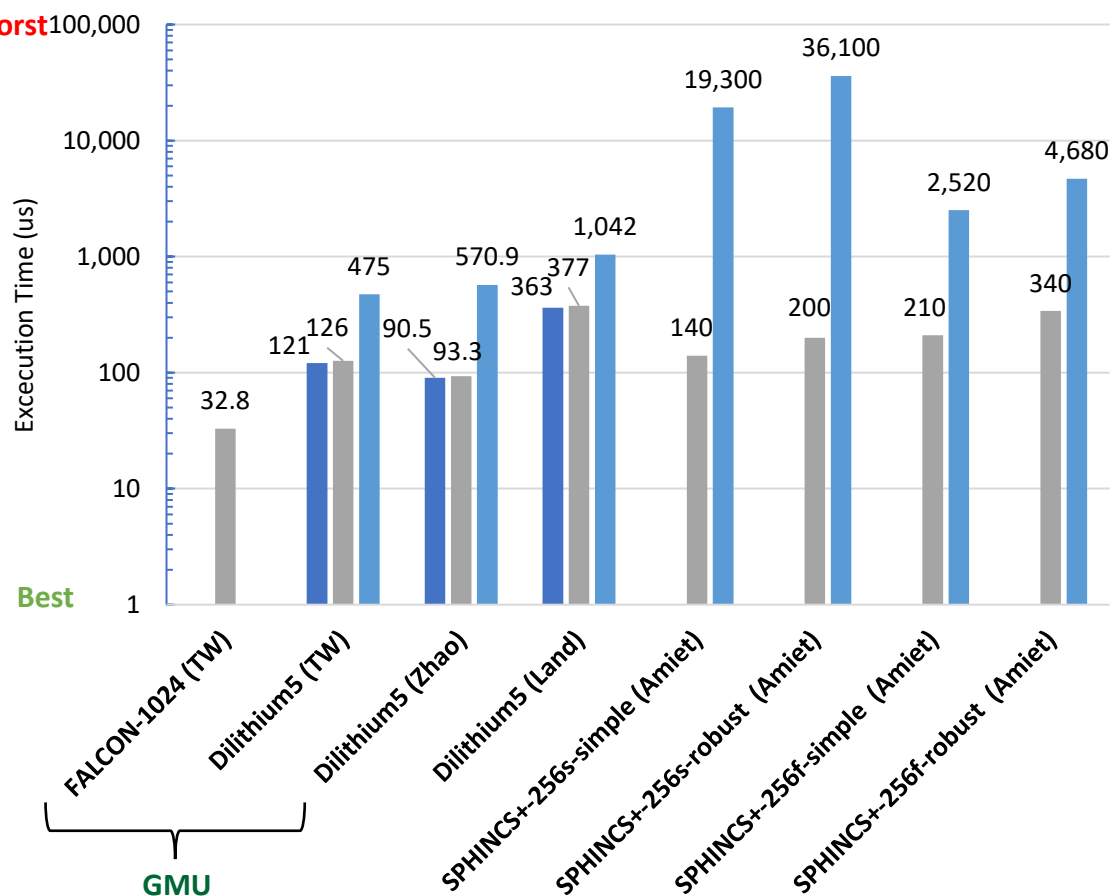
Result and Comparison - Context

Author	Algorithm	Optimization Target	Joint modules for each:		Keygen Included	FPGA Family
			operation?	security level?		
This Work	FALCON	High Performance	No	No	No	Artix-7, Kintex-7, Virtex UltraScale+
	Dilithium	High Performance	Yes	Yes	Yes	
Zhao et al (2022)	Dilithium	High Performance	Yes	Yes	Yes	Artix-7
Land et al (2021)	Dilithium	Balanced	Yes	No	Yes	Artix-7
Kales et al (2020)	Picnic	High Performance	Yes	No	No	Kintex-7,
Amiet et al (2020)	SPHINCS+	High Performance	Yes	No	No	Artix-7

FPGA Execution time + Resources – Artix-7

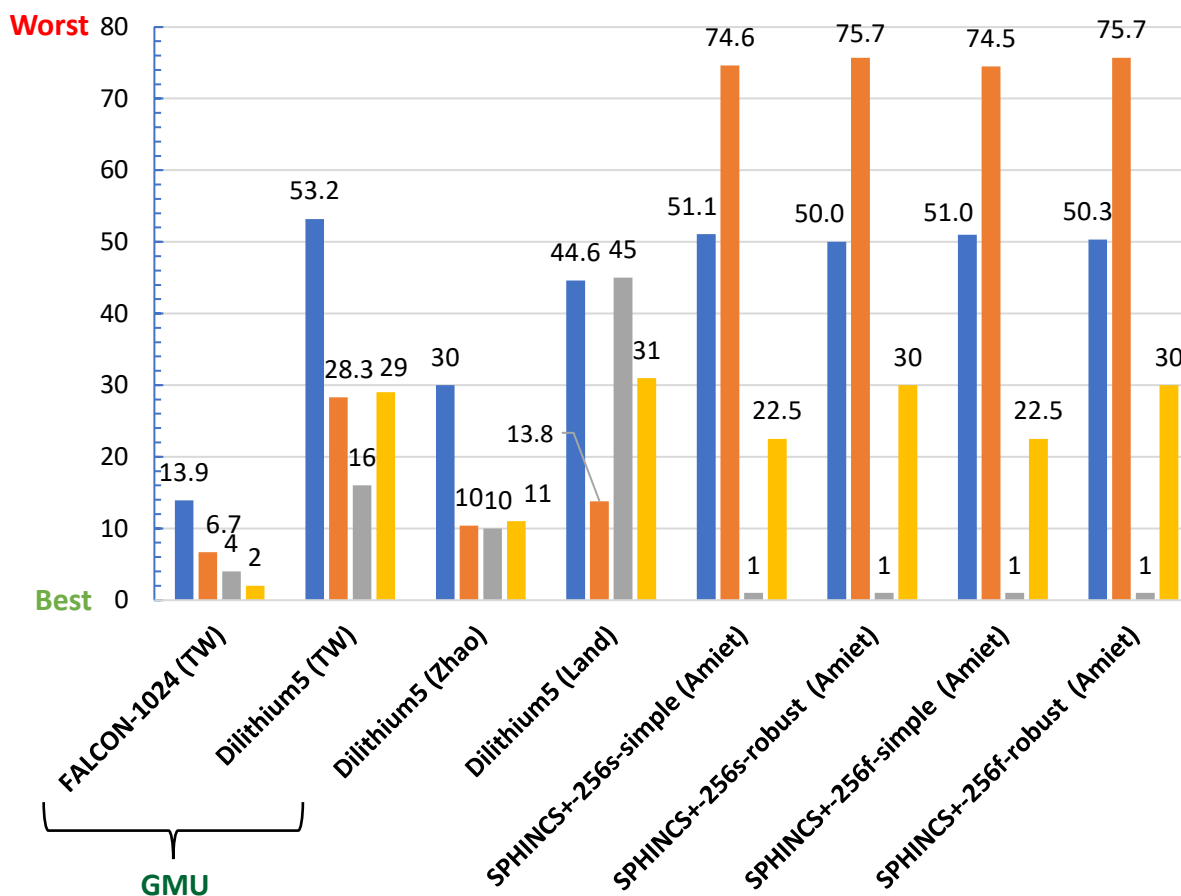
Execution Time (Artix-7, Security Level 5)

■ Keygen ■ Verify ■ Sign



Resource Utilization (Artix-7, Security Level 5)

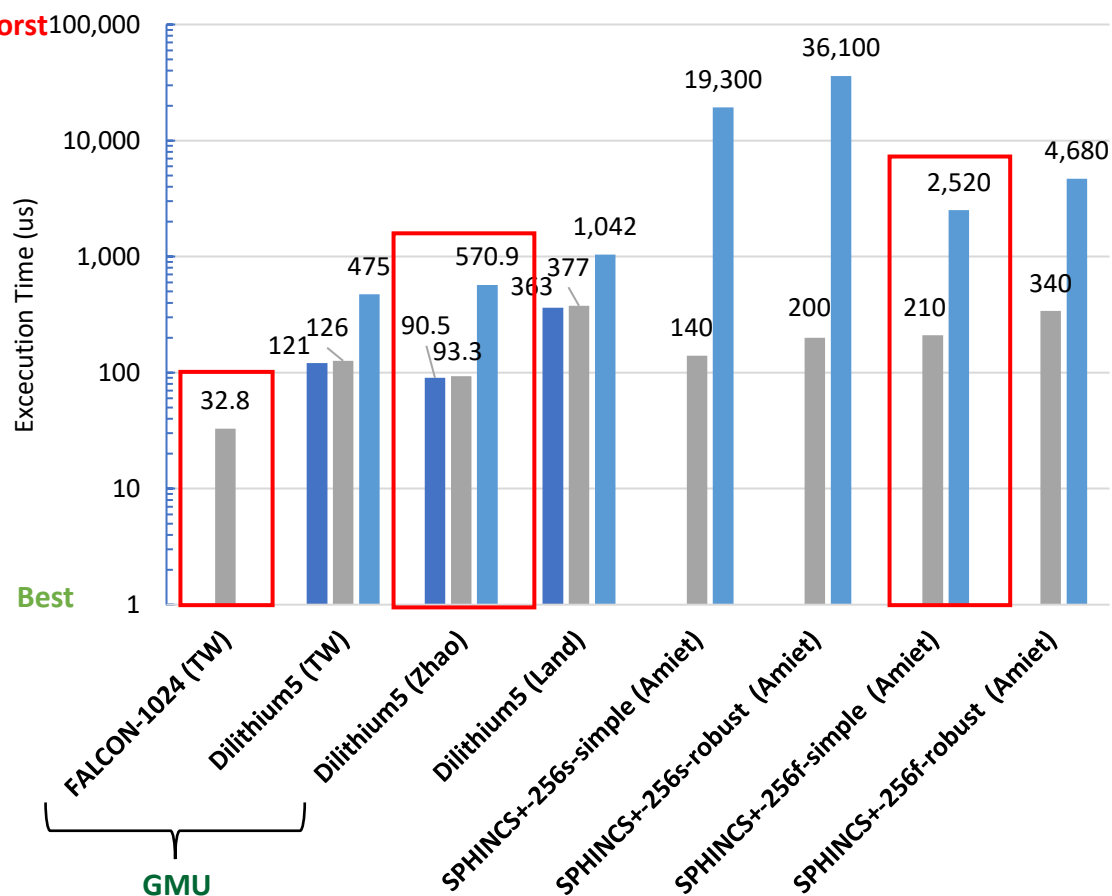
■ LUT (x1000) ■ FF (x1000) ■ DSP ■ BRAM



FPGA Execution time + Resources – Artix-7

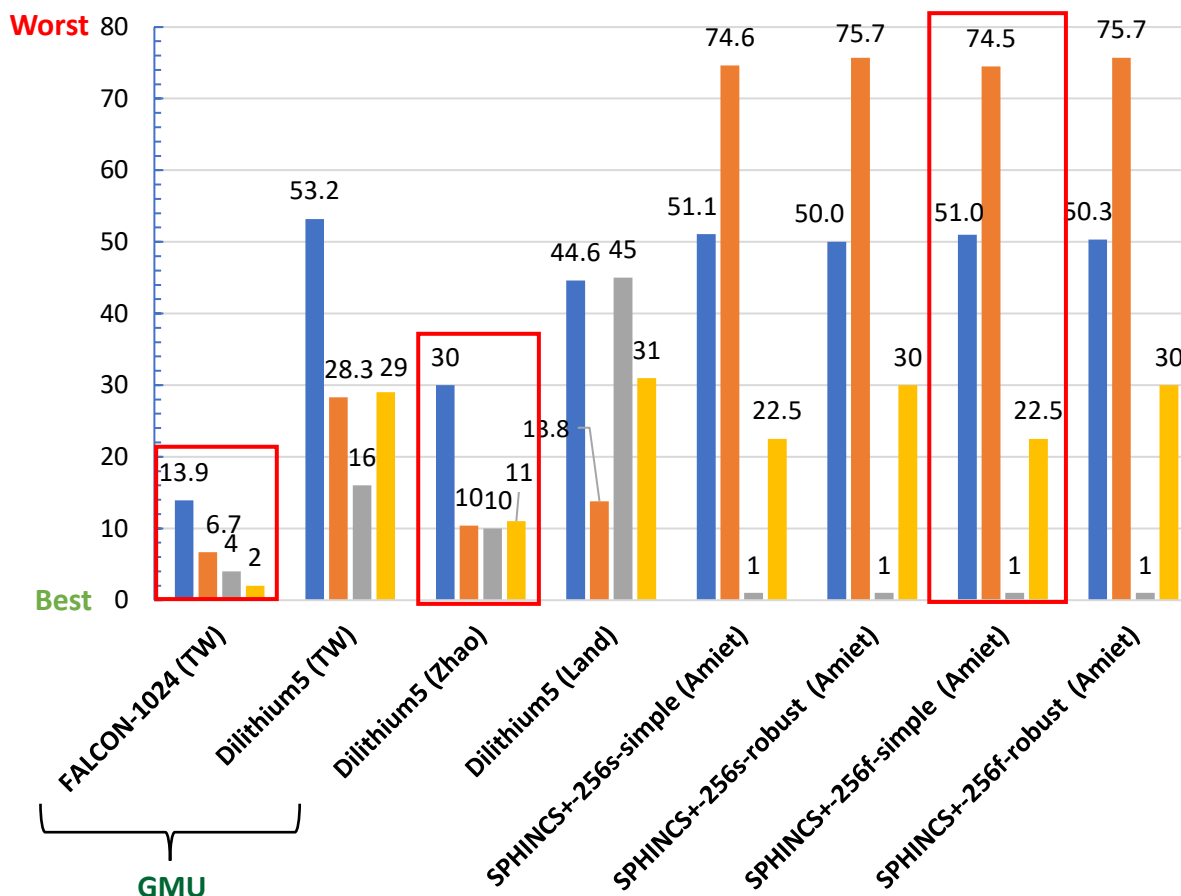
Execution Time (Artix-7, Security Level 5)

■ Keygen ■ Verify ■ Sign



Resource Utilization (Artix-7, Security Level 5)

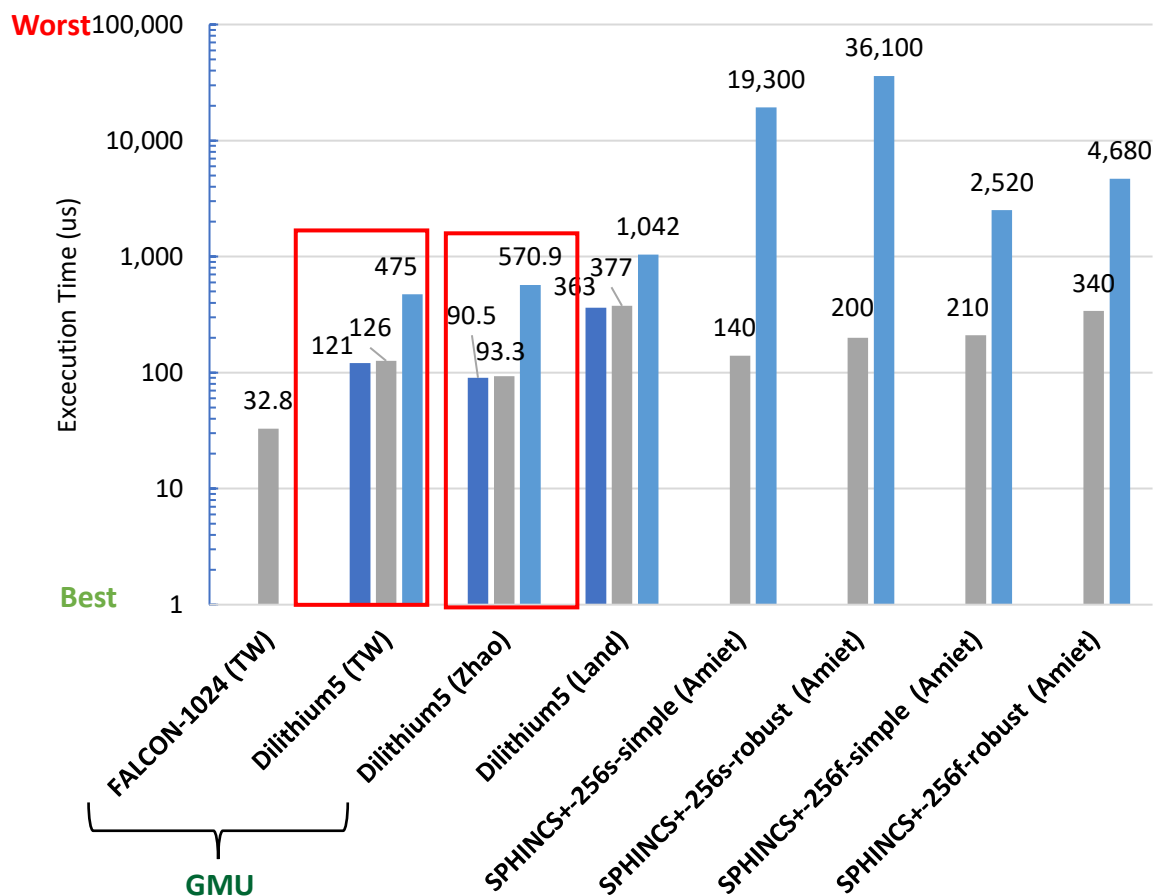
■ LUT (x1000) ■ FF (x1000) ■ DSP ■ BRAM



FPGA Execution time + Resources – Artix-7

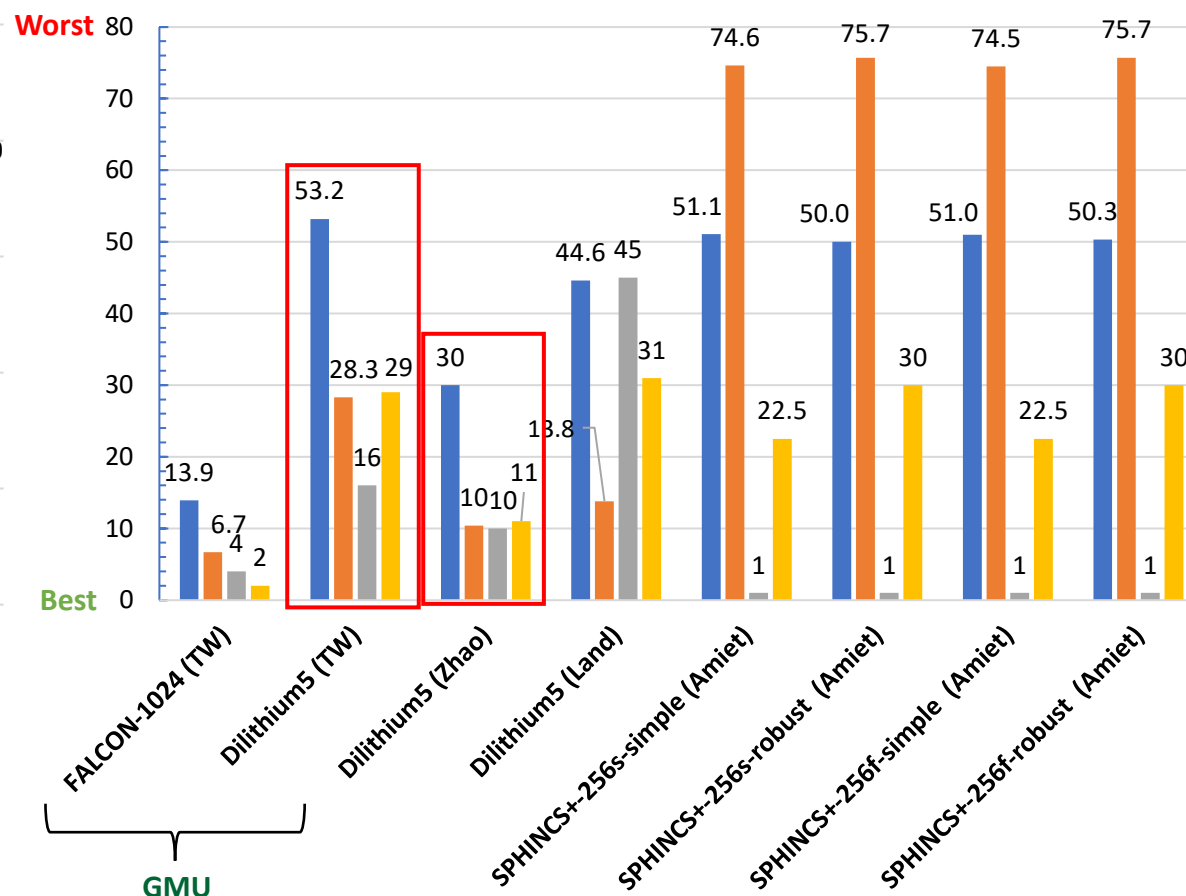
Execution Time (Artix-7, Security Level 5)

■ Keygen ■ Verify ■ Sign



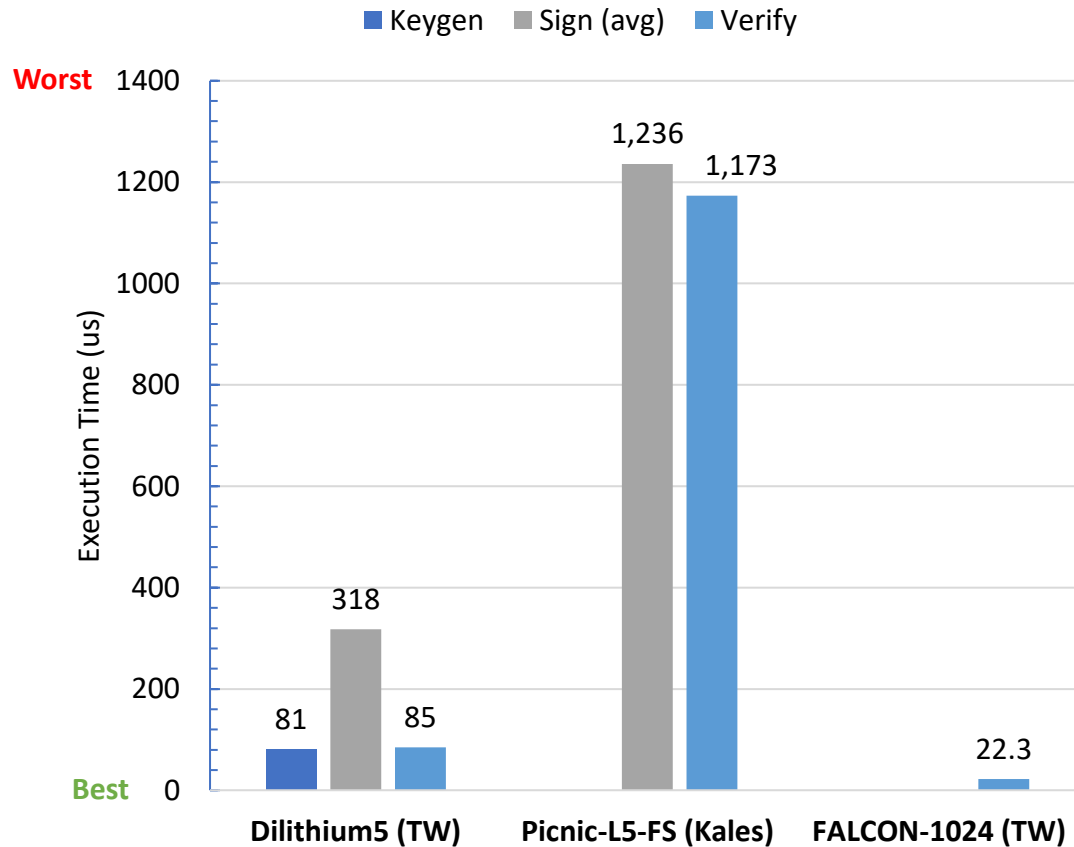
Resource Utilization (Artix-7, Security Level 5)

■ LUT (x1000) ■ FF (x1000) ■ DSP ■ BRAM

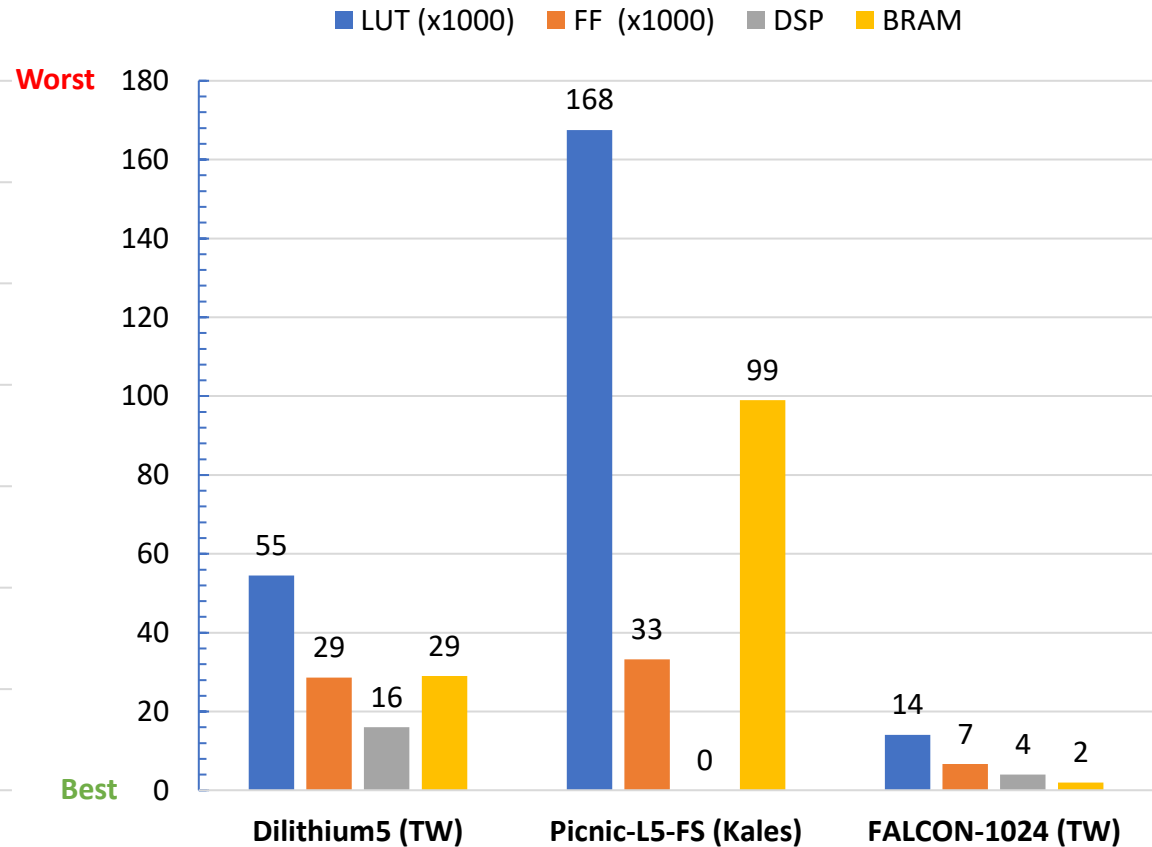


FPGA Execution time + Resources – Kintex-7

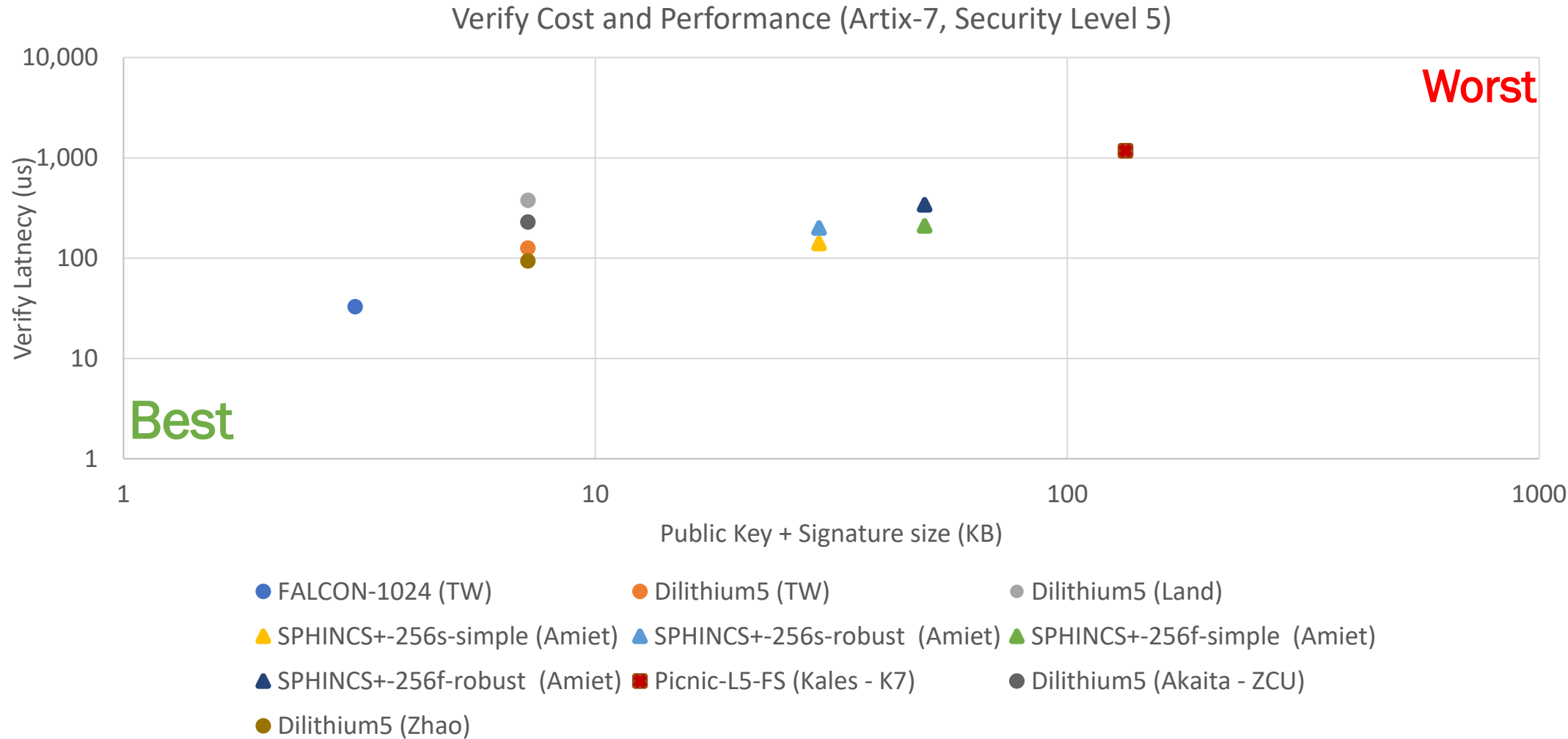
Execution Time (Kintex-7, Security Level 5)



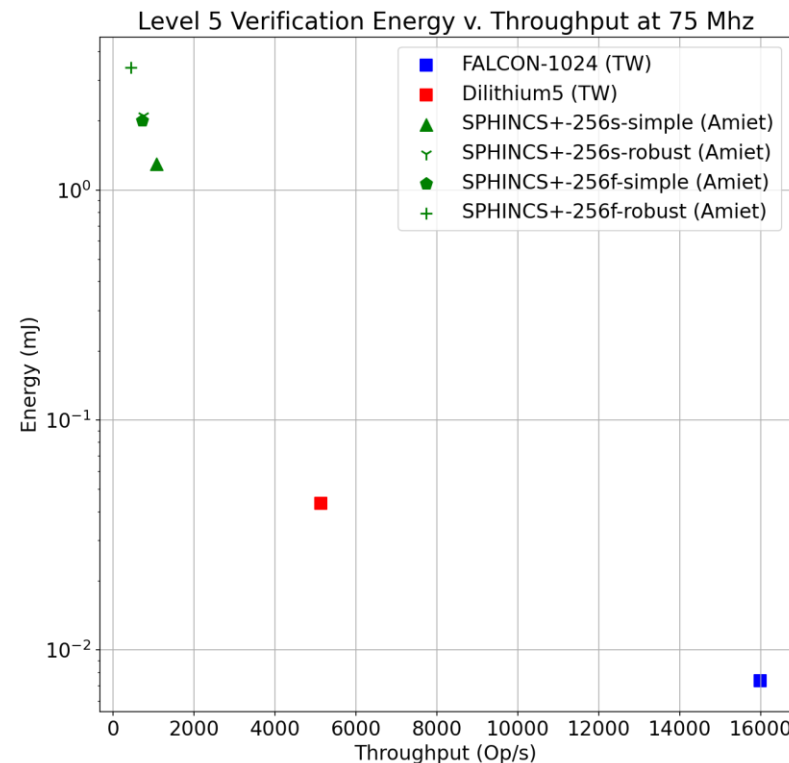
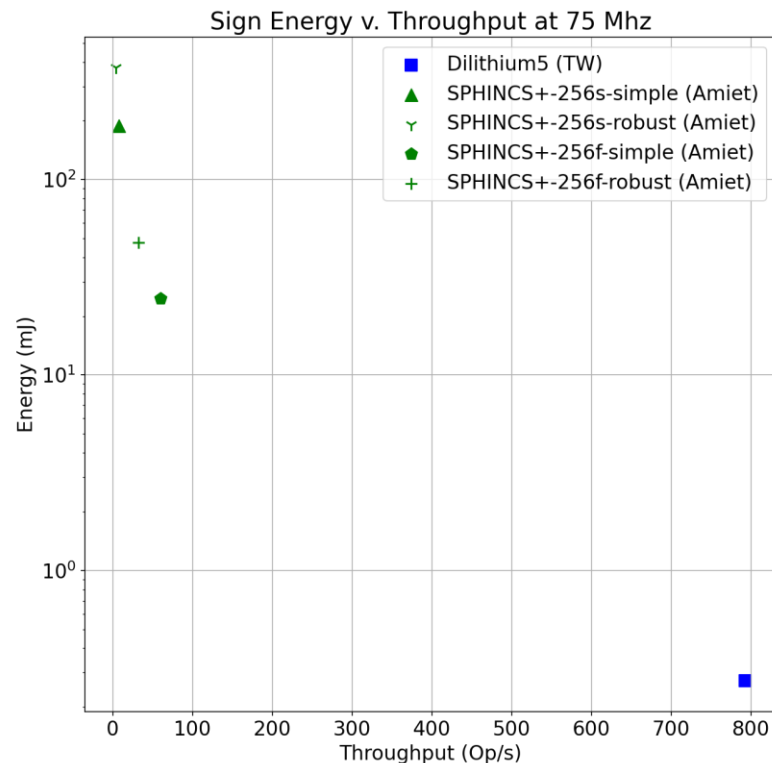
Resource Utilization (Kintex-7, Security Level 5)



Verification Comparison



Energy Consumption on Artix-7



More results in our second paper accepted without presentation:

<https://csrc.nist.gov/Events/2022/fourth-pqc-standardization-conference>

“FPGA Energy Consumption of Post-Quantum Cryptography”

Lessons Learned From Recent Work By Zhao et al

- **Pipeline Operations**
 - Passing polynomials directly between modules – **less BRAM needed for intermediate results**
- **On-the-Fly Sampling**
 - Re-sampling the A matrix for sign attempts – **no need to stored entire matrix for multiple sign attempts**
 - Enabled by a low-latency SHA-3 implementation rather than multiple slower instances
- **Carefully Optimize Submodules**
 - Lots of minor submodules are needed – **their area adds up!**

Conclusions

- **Main Contributions:**

- Developed high performance Dilithium hardware architecture with publicly available code: <https://github.com/GMUCERG/Dilithium>
- First FALCON implementation in hardware

- **Observations:**

- FALCON is the optimal algorithm based on **verify performance and transmission overhead**
- Dilithium is a **strong all-around** algorithm
- Symmetric-based algorithm are slower and have larger transmission sizes, **only the best from the perspective of maturity of security analysis**

Questions?

Thank you for your time!